Sardar Vallabhbhai National Institute of Technology, Surat
Department of Artificial Intelligence
Integrated B. Tech. (AI) / M. Tech. (AI) – Semester V
Soft Computing (IA305) – Laboratory
Lab 8: Artificial Neural Network – Multilayer Perceptron Trained by Backpropagation

Name:	**Roll No:**

Aim

To study the multilayer feed-forward neural network and implement the backpropagation algorithm to learn the XOR function, a mapping that a single-layer perceptron cannot represent.

Objective

- Understand forward propagation through a hidden layer and the backward propagation of error by the chain rule, and how the weight update follows from the gradient of the squared error.
- Implement a 2–2–1 network in Python and analyse the effect of the number of hidden units, the learning rate and the weight initialisation on convergence.

Theory

A single-layer perceptron computes one hyperplane, so it can separate only linearly separable classes, and XOR is not one: no straight line puts (0,1) and (1,0) on one side and (0,0) and (1,1) on the other. A hidden layer of non-linear units builds an intermediate representation in which the four patterns do become separable, and backpropagation trains the network by gradient descent on the squared error, propagating that error from the output layer back to the hidden layer through the chain rule.

- Forward pass: $a_1 = f(x \cdot W_1 + b_1)$, $a_2 = \sigma(a_1 \cdot W_2 + b_2)$, with the sigmoid $\sigma(z) = 1 / (1 + e^{-z})$ and σ' expressed through its own output as $a(1 - a)$.
- Error: $E = \tfrac{1}{2} \Sigma (a_2 - y)^2$, reported per epoch as the mean squared error over the four patterns.
- Deltas: $\delta_2 = (a_2 - y) \cdot a_2(1 - a_2)$ at the output, and $\delta_1 = (\delta_2 \cdot W_2^{T}) \cdot f'(a_1)$ at the hidden layer — the error of a hidden unit is the output error weighted by its own contribution to it.
- Updates: $W_2 \leftarrow W_2 - \eta \cdot a_1^{T} \delta_2$ and $W_1 \leftarrow W_1 - \eta \cdot x^{T} \delta_1$, with the biases updated by the column sums of the corresponding deltas; η is the learning rate, typically 0.1 to 1.0.
- XOR targets: $(0,0) \rightarrow 0$, $(0,1) \rightarrow 1$, $(1,0) \rightarrow 1$, $(1,1) \rightarrow 0$.

Algorithm

A. Initialization

1. Set the architecture — n inputs, h hidden units, m outputs — the learning rate η, the maximum number of epochs and the tolerance on the mean squared error at which training is considered converged.
2. Initialise W_1 and W_2 to small random values in $[-1, 1]$ and the biases to zero; the values must be random, since identical hidden units receive identical updates and never differentiate. Load the four XOR patterns as the training batch.

B. Training (repeat for each epoch)

1. Forward pass: compute the hidden activations and the output for all four patterns together, and record the mean squared error for the epoch.
2. Backward pass: compute the output delta and then the hidden delta, and update both weight matrices and both bias vectors using the learning rate.
3. Stop when the mean squared error falls below the tolerance and every output lies on the correct side of 0.5, or when the epoch limit is reached; report the epochs, the final error, the outputs and the learned weights.

Exercises

1. Vary the hidden units over 1, 2, 3, 4 and 8 and the learning rate over 0.1, 0.5 and 1.0, repeating each setting with twenty random seeds. Report how many of the twenty runs converge and the median epochs for each combination, state the smallest number of hidden units that can represent XOR at all, and say whether that number is reliable in practice.
2. Repeat the reference run with every weight initialised to zero instead of randomly, and separately with tanh in the hidden layer. Report convergence in each case, print the hidden weight matrix after training in the zero case, and explain what zero initialisation does to the hidden units.

Result

Record the epochs to convergence, the final mean squared error and the four outputs for the reference run, together with the two exercise tables, and comment on the settings at which training fails.